



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 7: Bubble Sort Algorithm & Bit Manipulation (Shift Instructions)

1. Objectives

- Implement and analyze the **Bubble Sort algorithm** in assembly.
- Explore **bit manipulation instructions** (shifting and rotating).
- Strengthen logic-building using low-level programming concepts.

2. Bubble Sort Algorithm

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. It gets its name because smaller or larger elements "bubble" to the top of the list.

2.1. Sorting Program

The following program sorts **10 numbers** stored in the data array using **Bubble Sort**.

```
[org 0x0100]
jmp start

data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0
swap: db 0 ; Flag to track if any swap happened

start:
    mov bx, 0 ; Initialize array index (BX = 0)
    mov byte [swap], 0 ; Reset swap flag = no swaps yet

loop1:
    mov ax, [data+bx] ; Load current number into AX
    cmp ax, [data+bx+2] ; Compare with next number
    jbe noswap ; Jump if already in correct order (<=)

    ; Swap numbers
    mov dx, [data+bx+2] ; Load second element into DX
    mov [data+bx+2], ax ; Put first element in second position
    mov [data+bx], dx ; Put second element in first position
    mov byte [swap], 1 ; Mark that a swap has been done
```



noswap:

```
    add bx, 2                ; Move to next element (word = 2 bytes)
    cmp bx, 18 ;Last index? (18 = second last element of 10 words)
    jne loop1                ; If not, repeat inner loop

    cmp byte [swap], 1      ; Check if any swap happened
    je start                 ; If yes, do another pass
mov ax, 0x4c00              ; Exit program
int 0x21
```

2.2. How It Works

1. Numbers are stored in data (10 words).
2. A **swap flag** is used to check if swapping happened in a pass
 - If no swap occurred, means array is already sorted → program exits
 - If swaps occurred → repeat another pass
3. Each pass/iteration compares **two adjacent numbers**, swaps if needed
4. Loop continues until sorted

2.3. Dry Run Example

Initial: 60, 55, 45, 50, 40, 35, 25, 30, 10, 0

- Pass 1 → 55, 45, 50, 40, 35, 25, 30, 10, 0, 60
- Pass 2 → 45, 50, 40, 35, 25, 30, 10, 0, 55, 60
- Pass 3 → 45, 40, 35, 25, 30, 10, 0, 50, 55, 60
- ... continues until → 0, 10, 25, 30, 35, 40, 45, 50, 55, 60

3. Bit Manipulation Instructions

Bit manipulation involves working with individual bits in binary numbers. Bit manipulation instructions are fundamental to low-level programming. They allow us to shift, and examine individual bits within a register or memory location.

Instruction	Name	Effect on Bits	Carry Flag (CF)	Flag Impact
SHR	Shift Right Logical	Bits shift right; LSB=0, MSB filled with 0. Unsigned divide by 2^{count} .	Old LSB	CF,OF
SHL	Shift Left Logical	Bits shift left; MSB=0, LSB filled with 0. Multiply by 2^{count} .	Old MSB	CF,OF
SAL	Shift Arithmetic Left	Same as SHL (arithmetic for signed, but identical in effect).	Old MSB	CF,OF
SAR	Shift Arithmetic Right	Bits shift right; LSB=0, MSB filled with sign bit (0 or 1). Signed divide by 2^{count} .	Old LSB	CF,OF



3.1. Shift Instructions

- **SHR (Shift Logical Right):** Divides by 2. Inserts 0 on left.
- **SHL (Shift Logical Left):** Multiplies by 2. Inserts 0 on right.
- **SAL (Shift Arithmetic Left):** Same as SHL.
- **SAR (Shift Arithmetic Right):** Preserves sign bit while shifting.

3.2. Program

```
[org 0x0100]
    mov al, 10                ; AL = 00001010
    shl al, 1                 ; AL = 00010100 (20)
    shr al, 1                 ; AL = 00001010 (10)

    mov al, -11               ; AL = 11110101 (signed 8-bit)
    sar al, 1                 ; AL = -6 (sign preserved)
mov ah, 0x4c
int 0x21
```

4. Lab Tasks

Task 1: Modify the bubble sort to:

- Sort in descending order
- Count number of swaps performed

Task 2: Bit Manipulation

1. Write a program that:
 - Takes a 16-bit number as input.
 - Moves the upper 8 bits to the lower position using right shifts.
 - Moves the lower 8 bits to the upper position using left shifts.
 - Stores both shifted results separately.
 - Then perform:
 - Left shift by 4 bits
 - Arithmetic right shift by 4 bits
2. Write a program that:
 - Creates a bit mask for a given bit position by shifting 1 left.
 - Shifts the mask right to move it back to lower bits.

Task 3: Write a program that:

1. Sorts an array using bubble sort
2. For each sorted element, perform:
 - Left shift by its position
 - Logical right shift by 3
 - Arithmetic right shift by 1 to adjust sign if needed